ASSIGNMENT– 9.5

2303A51655
Batch:29

Problem 1: String Utilities Function
ConsiderthefollowingPythonfunction:
defreverse_string(text):
returntext[::-1]
Task:
1. Writedocumentationin:
o (a) Docstring
o (b)Inline comments
o (c)Google-styledocumentation
2. Comparethethreedocumentationstyles.
3. Recommendthemostsuitablestyleforautility-basedstring library.

task1.py > ...

```python
def reverse_string(text):
    """Reverses the input string.

    Args:
        text (str): The string to reverse.

    Returns:
        str: The reversed string.
    """
    return text[::-1]


# ---- User Input ----
user_text = input("Enter a string to reverse: ")
result = reverse_string(user_text)
print("Reversed String:", result)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment 9.5.py/task1.py"
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task1.py"
Enter a string to reverse: cha6an
Reversed String: na6ahc
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Observation:

1. Docstringsprovideacleardescriptionofthefunctionanditspurpose.
2. Inlinecommentsexplainthelogicstep-by-stepinsidethecode.
3. Google-styledocumentationoffersastructured andstandardizedformat.
4. Inlinecomments areusefulforbeginnersbutnotsuitableforautomateddocumentation.
5. Google-styledocstringsarebestforutilitylibrariesbecausetheysupportdocumentation tools.

Problem2:PasswordStrengthChecker
Consider the function:
defcheck_strength(password):
returnlen(password)>=8 Task:
1. Documentthefunctionusingdocstring,inlinecomments,and Google style.
2. Comparedocumentationstylesforsecurity-relatedcode.
3. Recommendthemostappropriatestyle.

task2.py > ...

```python
def check_strength(password):
    """Checks if password length is at least 8 characters.

    Args:
        password (str): Password entered by user.

    Returns:
        bool: True if strong, False otherwise.
    """
    return len(password) >= 8


# ---- User Input ----
user_password = input("Enter your password: ")

if check_strength(user_password):
    print("Password is strong.")
else:
    print("Password is weak. Must be at least 8 characters.")
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task2.py"
Enter your password: armdgufiie
Password is strong.
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Observation:

1. Security-relatedfunctionsrequireclear andstructured documentation.
2. Docstringshelpexplainthepurposeandreturnvaluesofthefunction.
3. Inlinecommentsclarifythevalidationlogicusedinthefunction.
4. Google-styledocumentation improvesreadabilityandmaintainability.
5. Forsecuritymodules,Google-styledocumentationis recommendedduetoitsclarityand professional standard.

Problem3:MathUtilitiesModule Task:
1. Createamodulemath_utils.pywith functions:
o square(n)
o cube(n)
o factorial(n)
2. GeneratedocstringsautomaticallyusingAItools.
3. ExportdocumentationasanHTML file.

task3.py > ...

```python
def square(n):
    return n * n


def cube(n):
    return n * n * n


def factorial(n):
    if n < 0:
        raise ValueError("Factorial not defined for negative numbers")

    if n == 0 or n == 1:
        return 1

    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


# ---- User Input ----
num = int(input("Enter a number: "))

print("Square:", square(num))
print("Cube:", cube(num))
print("Factorial:", factorial(num))
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task3.py"
Enter a number: 34
Square: 1156
Cube: 39304
Factorial: 295232799039604140847618609643520000000
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Ln 27, Col 36 Spaces: 4 UTF-8

Observation:

- Docstringsallowautomaticdocumentationgenerationusingtoolslike pydoc.
- Properdocumentationhelpsdevelopersunderstandmathematicalfunctionseasily.
- HTMLdocumentationmakesmodulesaccessible throughbrowsers.
- Structureddocumentationimprovesthemaintainabilityofutilitymodules.
- Usingstandardizeddocumentationpracticesensuresconsistencyacrossthe project.

Problem4:AttendanceManagementModule

Task:

1. Createamoduleattendance.pywith functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)

2. Addproperdocstrings.

3. Generateandviewdocumentationinterminalandbrowse

task4.py > ...

```python
attendance_record = {}

def mark_present(student):
    attendance_record[student] = "Present"

def mark_absent(student):
    attendance_record[student] = "Absent"

def get_attendance(student):
    return attendance_record.get(student, "Not Recorded")


# ---- User Input ----
while True:
    print("\n1. Mark Present")
    print("2. Mark Absent")
    print("3. Get Attendance")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        name = input("Enter student name: ")
        mark_present(name)
        print("Marked Present.")

    elif choice == "2":
        name = input("Enter student name: ")
        mark_absent(name)
        print("Marked Absent.")

    elif choice == "3":
        name = input("Enter student name: ")
        print("Attendance Status:", get_attendance(name))

```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task4.py"

1. Mark Present
2. Mark Absent
3. Get Attendance
4. Exit
Enter your choice: 1
Enter student name: ram
Marked Present.
```

Observation:

1. Documentationhelpsclarifythepurposeofeachattendance function.
2. Docstringsallowtoolstoautomaticallygeneratemoduledocumentation.
3. Properlydocumentedmodulesare easier forteamstomaintain.
4. Terminalandbrowserdocumentationimprovesaccessibilityfordevelopers.
5. Cleardocumentationensuresaccurateusage ofattendance functions.

Problem5:FileHandlingFunction

Consider the function:

defread_file(filename):

withopen(filename,'r')asf:

returnf.read()

Task:

1. Writedocumentationusingallthreeformats.

2. Identifywhichstylebestexplainsexception handling.

3. Justifyyour recommendation.

```python
task5.py > ...
def read_file(filename):
    """Reads contents of a file."""
    with open(filename, 'r') as f:
        return f.read()


# ---- User Input ----
file_name = input("Enter file name to read: ")

try:
    content = read_file(file_name)
    print("\nFile Content:\n")
    print(content)

except FileNotFoundError:
    print("Error: File not found.")

except PermissionError:
    print("Error: Permission denied.")

except Exception as e:
    print("Unexpected error:", e)
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task5.py"
Enter file name to read: ram
Error: File not found.
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Observation:

1. Filehandlingfunctionsrequiredocumentationforparametersandreturneddata.
2. Exceptionhandlingmustbeclearlydescribedinthedocumentation.
3. Inlinecommentshelpexplainfileoperationsstep-by-step.
4. Docstringsallowdocumentationtoolstodisplayfunctiondetailsautomatically.
5. Google-styledocumentationisbestforexplainingexceptionsandfileoperationsclearly.